A Project Report On

Insurance Policy Prediction

Submitted in partial fulfillment of the requirement for the award of the degree

MASTER OF COMPUTER APPLICATION from

Marwadi University

Academic Year 2026 – 27

Milan Jethva (92500584043)
Shrey Palan (92500584049)

Internal Guide
Prof. Gehna Sachdeva



Rajkot-Morbi Road, At & PO : Gauridad, Rajkot 360 003. Gujarat. India.

Marwadi University
Marwadi Chandarana Group

NAAC A+

Faculty of Computer Applications (FoCA)

Certificate

This is to certify that the project work entitled Insurance Policy Prediction Using classification algorithms submitted in partial fulfillment of the requirement for the award of the degree of

Master of Computer Application

of the

Marwadi University

is a result of the bonafide work carried out by

Milan Jethva (92500584043)
Shrey Palan (92500584049)

during the academic year 2026 – 2027

Porf. Gehna Sachdeva	*Dr. Sunil Bajeja*	*Dr. Rijwan Khan*
Faculty Guide	**HOD**	**Dean**

DECLARATION

We hereby declare that this project work **Insurance Policy Prediction Using classification algorithms** is a record done by us.

We also declare that the matter embodied in this project is genuine work done by us and has not been submitted whether to this University or to any other University / Institute for the fulfillment of the requirement of any course of study.

Place: Rajkot Date:

Milan Jethva (92500584043) Signature: _____
Shrey Palan (92500584049) Signature: _____

ACKNOWLEDGEMENT

It is indeed a great pleasure to express our thanks and gratitude to all those who helped us. No serious and lasting achievement or success one can ever achieve without the help of friendly guidance and co-operation of so many people involved in the work.

We are very thankful to our guide **Prof. Gehna Sachdeva,** the person who makes us to follow the right steps during our project work. We express our deep sense of gratitude to for his guidance, suggestions and expertise at every stage. A part from that his valuable and expertise suggestion during documentation of our report indeed help us a lot.

Thanks to our friend and colleague who have been a source of inspiration and motivation that helped to us during our project work.

We are heartily thankful to the Dean of our department **Dr. Rijwan Khan** for giving us an opportunity to work over this project and for their end-less and great support to all other people who directly or indirectly supported and help us to fulfil our task.

Milan Jethva (92500584043) Signature: _____
Shrey Palan (92500584049) Signature: _____

CONTENTS

Chapters	Particulars	Page No.
1	**Introduction** 1.1. Objective of the New System 1.2. Problem Definition 1.3. Core Components 1.4. Project Profile 1.5. Assumptions and Constraints 1.6. Advantages and Limitations of the Proposed System	6
2	**Requirement Determination & Analysis** 2.1. Requirement Determination 2.2. Targeted Users 2.3. Tool details (Python / PowerBI/ Tableau) 2.4. Library description (Details on various libraries / packages used)	7
3	**System Design** 3.1. Flowchart / Algorithm with steps 3.2. Dataset Design 3.3. Details on preprocessing steps applied	8
4	**Development** 4.1 Script details / Source code and Screenshots 4.2. Test reports	10
5	**Proposed Enhancements**	20
6	**Conclusion**	21
7	**Bibliography**	21

1. Introduction

1.1 Objective of the New System:

The insurance industry plays an important role in providing financial protection against various risks and uncertaianties. Insurance companies need to understand customer characteristics and behavior to identify customers who are likely to purchase an insurance policy.

This project, **Comparative Analysis of Machine Learning Classification Algorithms for Insurance Policy Prediction,** uses machine learning techniques to predict whether a customer will take an insurance policy or not. The dataset contains customer-related information such as Age, Gender, Annual Income, Marital Status, City, Occupation, Smoking Status, Existing Disease, Number of Dependents, Policy Term Years, Health Score, and Premium Amount.

1.2 Problem Definition:

Insurance companies handle large amounts of customer data, and manually analyzing this data to identify potential insurance customers can be difficult and time-consuming.

The main problem addressed in this project is: How can machine learning classification algorithms be used to predict whether a customer will take an insurance policy based on their personal, financial, and health-related information?

1.3 Core Components:

- **Data Collection:** The insurance dataset is loaded and used for analysis and model development.
- **Data Cleaning:** The dataset is checked for missing values and duplicate records.
- **Data Preprocessing:** Categorical data is converted into numerical format using Label Encoding. The dataset is also prepared for machine learning algorithms.
- **Feature and Target Selection:** Input features are separated from the target variable, Insurance_Taken. The Policy_Status column is removed to avoid data leakage.
- **Train-Test Split:** The dataset is divided into training and testing data.
- **Feature Scaling:** Numerical feature values are scaled using StandardScaler where required.
- **Model Training:** Multiple classification algorithms are trained:
  - Logistic Regression
  - Decision Tree
  - Random Forest
  - KNN
  - SVM

- Naive Bayes

- **Model Evaluation:** The models are evaluated using:
  - Accuracy
  - Precision
  - Recall
  - F1-Score
  - Classification Report
  - Confusion Matrix

- **Model Comparison:** The performance of all algorithms is compared to identify the best model.

1.4 Project Profile:

Project Name	**Comparative Analysis of Machine Learning Classification Algorithms for Insurance Policy Prediction**
Project Type	Classification
Target Variable	Insurance_Taken
Classification Type	Binary Classification
Dataset	Insurance Customer Dataset
Main Language	Python
Data Analysis Libraries	Pandas, NumPy
Visualization Libraries Library	Matplotlib, Seaborn
Machine Learning	Scikit-learn
Algorithms Used	Logistic Regression, Decision Tree, Random Forest, KNN, SVM, Naive Bayes
Evaluation Metrics	Accuracy, Precision, Recall, F1-Score, Confusion Matrix

1.5 Assumptions and Constraints:

- **Assumptions:**
  - The dataset contains accurate and relevant customer information.
  - The Insurance_Taken column correctly represents whether a customer has taken insurance.
  - The selected features have a meaningful relationship with the insurance decision.
  - The training data is representative of the customer data used for prediction.

- **Constraints:**
  - The prediction accuracy depends on the quality of the dataset.
  - The model is limited to the features available in the dataset.
  - New or unseen customer behavior may differ from the training data.
  - Some algorithms may require more computational resources and training time.

1.6 Advantages and Limitations of the Proposed System:

- **Advantages:**
  - Automates the prediction of insurance policy decisions.
  - Analyzes a large number of customer records efficiently.
  - Compares multiple machine learning algorithms.
  - Helps identify the best-performing classification model.

- **Limitations:**
  - Model performance depends on data quality.
  - Limited features may affect prediction performance.
  - The model may not perform equally well on completely new or different customer data.
  - Machine learning predictions are not always 100% accurate.

2. Requirement Determination & Analysis

2.1 Requirement Determination:

The system requires appropriate software tools, machine learning libraries, and a suitable computing environment. The major requirements include:

- Insurance customer dataset
- Python programming language
- Data analysis and visualization tools
- Machine learning libraries
- Classification algorithms
- Model evaluation techniques

2.2 Targeted Users:

- **Insurance Companies:**
  Insurance companies can use the system to analyze customer data and understand the factors that may influence insurance policy decisions.

- **Data Analysts:**
  Data analysts can use the system to study customer information, identify patterns, and visualize important relationships within the dataset.

- **Data Scientists and Machine Learning Developers:**
  Data scientists and developers can use the system to experiment with different classification algorithms and evaluate their performance.

- **Students and Researchers:**
  Students and researchers can use the project to understand the practical application of data preprocessing, machine learning classification, model training, and performance evaluation.

2.3 Tool details:

Python:

Python is the primary programming language used in this project. It is widely used in data analysis, machine learning, and artificial intelligence because it is simple, flexible, and provides a large number of useful libraries.

In this project, Python is used for data processing, data analysis, visualization, machine learning model development, prediction, and performance evaluation.

Python provides an efficient environment for implementing and comparing multiple classification algorithms.

2.4 Library description (Details on various libraries / packages used):

Pandas:
Pandas is a Python library used for data manipulation and analysis. It helps in loading, organizing, cleaning, and processing structured datasets. It is useful for handling rows, columns, missing values, duplicate records, and other data-related operations.

NumPy:
NumPy is a Python library used for numerical and mathematical operations. It provides support for arrays and helps perform calculations efficiently.

Matplotlib:
Matplotlib is a data visualization library used to create different types of graphs and charts. It helps in presenting data in a visual and understandable form.

Seaborn:
Seaborn is a statistical data visualization library built on top of Matplotlib. It is used to create informative and visually clear graphs for analyzing relationships and patterns in data.

Scikit-learn:
Scikit-learn is one of the most widely used machine learning libraries in Python. It provides tools for data preprocessing, feature scaling, data splitting, machine learning model development, prediction, and performance evaluation.

3. System Design

3.1 Flowchart / Algorithm with steps:

Algorithm:

Step 1: Start

Step 2: Load and understand the insurance dataset.

Step 3: Perform data preprocessing by cleaning, encoding, and preparing the data.

Step 4: Split the dataset into training and testing data.

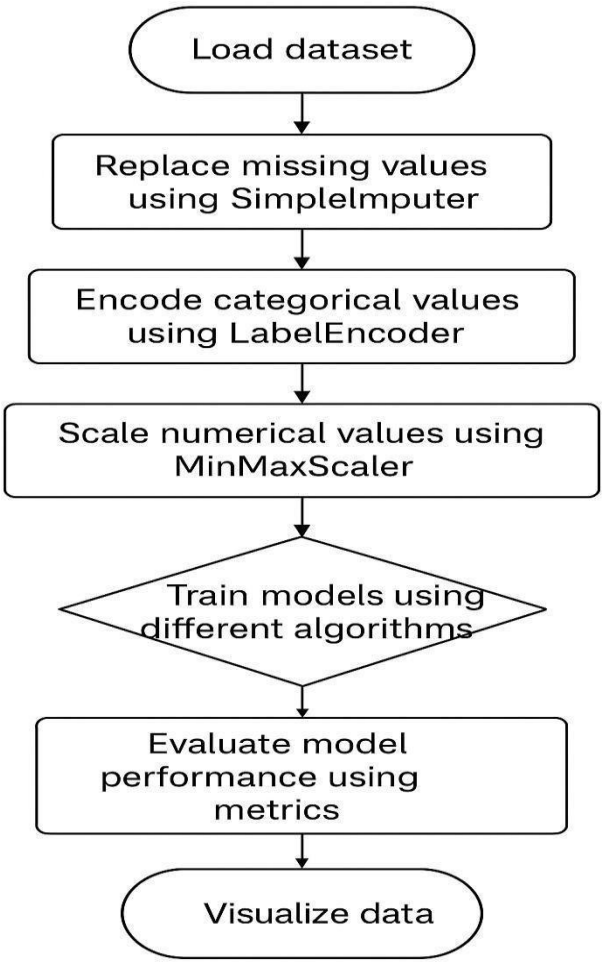
Step 5: Train classification algorithms such as Logistic Regression, Decision Tree, Random Forest, KNN, SVM, and Naive Bayes.

Step 6: Evaluate and compare the performance of all models.

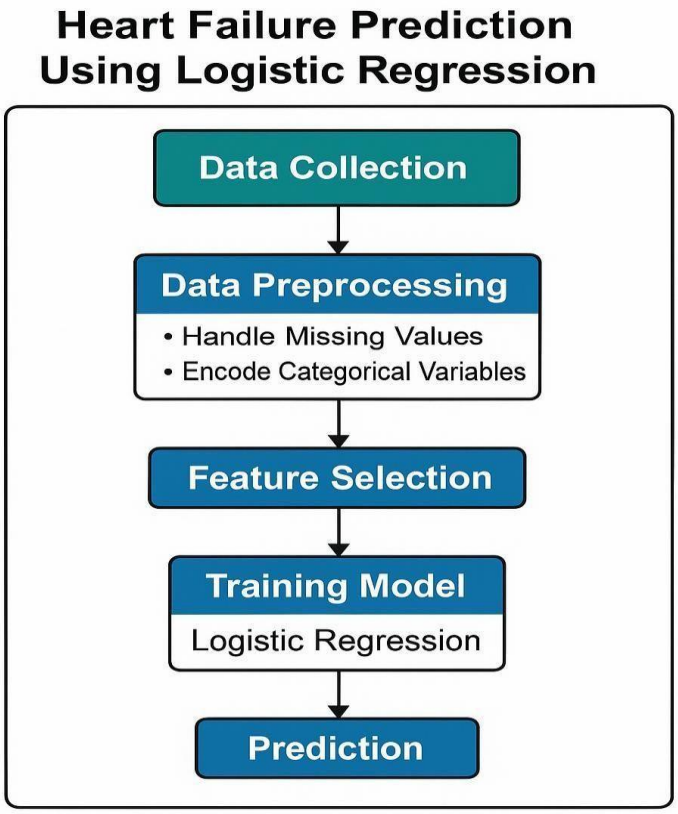
Step 7: Select the best-performing model and display the results.

Step 8: Stop

Flowchart:


Load dataset
Replace missing values using SimpleImputer
Encode categorical values using LabelEncoder
Scale numerical values using MinMaxScaler
Train models using different algorithms
Evaluate model performance using metrics
Visualize data

3.2 Dataset Design:



The dataset includes 50,000 synthetic samples with various attributes such as age, blood pressure, cholesterol levels, glucose levels, smoking habits, and heart disease status. The target variable is "HeartDisease".

3.3 Details on preprocessing steps applied:

- **Checking Missing Values:**
  The dataset was checked for missing or null values to ensure data quality. The dataset contained no missing values.

- **Checking and Removing Duplicate Records:**
  Duplicate records were checked and removed to avoid repeated data and improve the quality of the dataset.

- **Removing Data Leakage Column:**
  The Policy_Status column was removed because it contains information closely related to the target variable, Insurance_Taken. Keeping this column could lead to data leakage and unrealistic model performance.

- **Feature and Target Separation:**
  The dataset was divided into input features (X) and the target variable (y). The Insurance_Taken column was selected as the target variable.

- **Encoding Categorical Data:**
  Categorical features such as Gender, Marital Status, City, Occupation, Smoker, and Existing Disease were converted into numerical form using Label Encoding so that machine learning algorithms could process them.

- **Train-Test Split:**
  The processed dataset was divided into training and testing data. The training data was used to train the classification models, while the testing data was used to evaluate their performance.

- **Feature Scaling:**
  Feature scaling was applied using StandardScaler to bring numerical values into a similar range, which is particularly useful for algorithms such as KNN, SVM, and Logistic Regression.

4. Development

4.1 Script Details / Source Code and Screenshots

The project is divided into two main Python scripts:

Important Library:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder, StandardScaler

from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.neighbors import KNeighborsClassifier
from sklearn.svm import SVC
from sklearn.naive_bayes import GaussianNB

from sklearn.metrics import accuracy_score
from sklearn.metrics import precision_score
from sklearn.metrics import recall_score
from sklearn.metrics import f1_score
from sklearn.metrics import classification_report
from sklearn.metrics import confusion_matrix
```

```
df = pd.read_csv("insurance_40000_clean.csv")
```

```
df.head()
```

	Age	Gender	Annual_Income	Marital_Status	City	Occupation	Smoker	Existing_Disease	Dependents	Policy_Term_Years	Health_Score	Premium_Amount	Insurance_Taken	Pol
0	22	Female	1018618	Married	Chennai	Accountant	No	No	5	10	93.6	16627.54	Yes	
1	59	Female	856094	Divorced	Delhi	Private Employee	No	No	0	2	92.5	14495.68	Yes	
2	52	Male	202182	Married	Jaipur	Software Developer	No	No	5	10	84.3	23421.06	No	No
3	41	Female	460340	Married	Vadodara	Student	No	No	2	10	100.0	15636.25	No	No
4	40	Female	621088	Married	Jaipur	Teacher	Yes	No	4	2	86.7	19622.46	Yes	

Terminal

```
print("Shape:", df.shape)

print("\nColumns:")
print(df.columns)

print("\nDataset Information:")
df.info()

print("\nNull Values:")
print(df.isnull().sum())

print("\nDuplicate Rows:")
print(df.duplicated().sum())
```

```
Shape: (12981, 14)

Columns:
Index(['Age', 'Gender', 'Annual_Income', 'Marital_Status', 'City',
       'Occupation', 'Smoker', 'Existing_Disease', 'Dependents',
       'Policy_Term_Years', 'Health_Score', 'Premium_Amount',
       'Insurance_Taken', 'Policy_Status'],
      dtype='object')

Dataset Information:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 12981 entries, 0 to 12980
Data columns (total 14 columns):
 #   Column             Non-Null Count  Dtype
---  ------             --------------  -----
 0   Age                12981 non-null  int64
 1   Gender             12981 non-null  object
 2   Annual_Income      12981 non-null  int64
 3   Marital_Status     12981 non-null  object
 4   City               12981 non-null  object
 5   Occupation         12980 non-null  object
 6   Smoker             12980 non-null  object
 7   Existing_Disease   12980 non-null  object
 8   Dependents         12980 non-null  float64
 9   Policy_Term_Years  12980 non-null  float64
 10  Health_Score       12980 non-null  float64
 11  Premium_Amount     12980 non-null  float64
 12  Insurance_Taken    12980 non-null  object
 13  Policy_Status      12980 non-null  object
dtypes: float64(4), int64(2), object(8)
memory usage: 1.4+ MB

Null Values:
Age                 0
Gender              0
Annual_Income       0
Marital_Status      0
City                0
Occupation          1
Smoker              1
Existing_Disease    1
Dependents          1
Policy_Term_Years   1
Health_Score        1
Premium_Amount      1
Insurance_Taken     1
Policy_Status       1
dtype: int64

Duplicate Rows:
0
```

```
df = df.drop_duplicates()

print("Shape after removing duplicates:", df.shape)
```

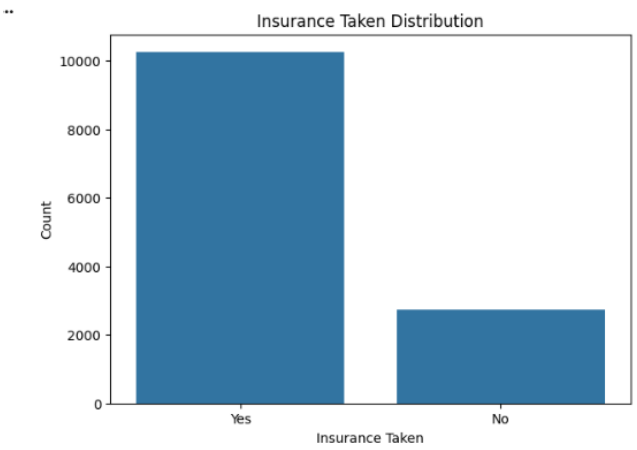
```
Shape after removing duplicates: (12981, 14)
```

```
plt.figure(figsize=(7, 5))

sns.countplot(data=df, x="Insurance_Taken")

plt.title("Insurance Taken Distribution")
plt.xlabel("Insurance Taken")
plt.ylabel("Count")

plt.show()
```




```
# Before removing
print("Columns Before Removing:")
print(df.columns.tolist())

# Remove Policy_Status
df = df.drop(columns=["Policy_Status"])

# After removing
print("\nColumns After Removing:")
print(df.columns.tolist())

# See dataset shape
print("\nDataset Shape After Removing:")
print(df.shape)

# Display first 5 rows
print("\nFirst 5 Rows:")
print(df.head())
```

```
Columns Before Removing:
['Age', 'Gender', 'Annual_Income', 'Marital_Status', 'City', 'Occupation', 'Smoker', 'Existing_Disease', 'Dependents', 'Policy_Term_Years', 'Health_Score',

Columns After Removing:
['Age', 'Gender', 'Annual_Income', 'Marital_Status', 'City', 'Occupation', 'Smoker', 'Existing_Disease', 'Dependents', 'Policy_Term_Years', 'Health_Score',

Dataset Shape After Removing:
(40000, 13)

First 5 Rows:
   Age  Gender  Annual_Income Marital_Status      City         Occupation  \
0   22  Female        1018618       Married   Chennai         Accountant
1   59  Female         856094      Divorced     Delhi    Private Employee
2   52    Male         202182       Married    Jaipur  Software Developer
3   41  Female         460340       Married  Vadodara            Student
4   40  Female         621088       Married    Jaipur            Teacher

  Smoker Existing_Disease  Dependents  Policy_Term_Years  Health_Score  \
0     No              No           5                 10          93.6
1     No              No           0                  2          92.5
2     No              No           5                 10          84.3
3     No              No           2                 10         100.0
4    Yes              No           4                  2          86.7

   Premium_Amount Insurance_Taken
0        16627.54             Yes
1        14495.68             Yes
2        23421.06              No
3        15636.25              No
4        19622.46             Yes
```

- **Training Model**

———— MODEL TRAINING ————

Feature Scaling in algorithm

- Logistic Regression
- KNN
- SVM
- Naive Byes

Algorithm1: Logistic Regression

- scaler = StandardScaler()
- X_train_scaled = scaler.fit_transform(X_train)
- X_test_scaled = scaler.transform(X_test)
- logistic_model = LogisticRegression(max_iter=5000)
- logistic_model.fit(X_train_scaled, y_train)
- y_pred_logistic = logistic_model.predict(X_test_scaled)

Evalution

```
print("Accuracy:",
      accuracy_score(y_test, y_pred_logistic))

print("Precision:",
      precision_score(y_test, y_pred_logistic))

print("Recall:",
      recall_score(y_test, y_pred_logistic))

print("F1 Score:",
      f1_score(y_test, y_pred_logistic))

print("\nClassification Report:\n")

print(classification_report(
    y_test,
    y_pred_logistic
))
```

```
Accuracy: 0.7904468412942989
Precision: 0.7904468412942989
Recall: 1.0
F1 Score: 0.882960413080895

Classification Report:

              precision    recall  f1-score   support

           0       0.00      0.00      0.00       544
           1       0.79      1.00      0.88      2052

    accuracy                           0.79      2596
   macro avg       0.40      0.50      0.44      2596
weighted avg       0.62      0.79      0.70      2596
```

Confusion Matrix

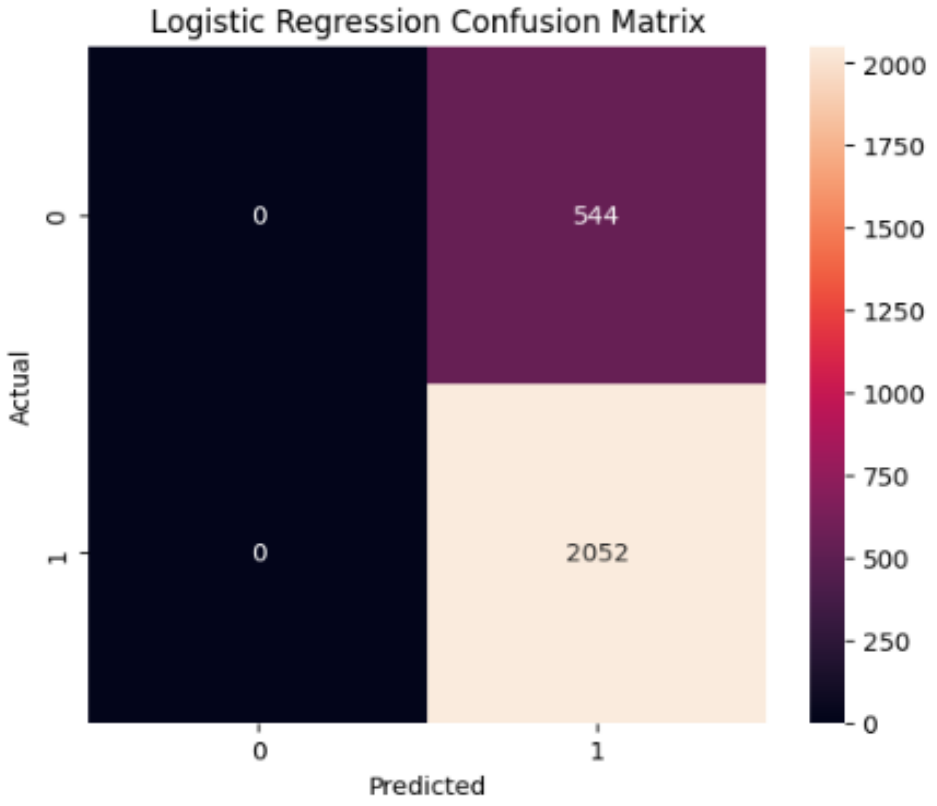
```
cm = confusion_matrix(y_test, y_pred_logistic)

plt.figure(figsize=(6, 5))

sns.heatmap(
    cm,
    annot=True,
    fmt="d"
)

plt.title("Logistic Regression Confusion Matrix")
plt.xlabel("Predicted")
plt.ylabel("Actual")

plt.show()
```



Algorithm2: KNN

```
knn_model = KNeighborsClassifier(
    n_neighbors=5
)

knn_model.fit(X_train_scaled, y_train)

y_pred_knn = knn_model.predict(X_test_scaled)
print("")
```

Evalution

```
print("Accuracy:",
      accuracy_score(y_test, y_pred_knn))

print("Precision:",
      precision_score(y_test, y_pred_knn))

print("Recall:",
      recall_score(y_test, y_pred_knn))

print("F1 Score:",
      f1_score(y_test, y_pred_knn))

print("\nClassification Report:\n")

print(classification_report(
    y_test,
    y_pred_knn
))
```

```
Accuracy: 0.7681047765793528
Precision: 0.8041107382550335
Recall: 0.9342105263157895
F1 Score: 0.8642921550946799

Classification Report:

              precision    recall  f1-score   support

           0       0.36      0.14      0.20       544
           1       0.80      0.93      0.86      2052

    accuracy                           0.77      2596
   macro avg       0.58      0.54      0.53      2596
weighted avg       0.71      0.77      0.73      2596
```

Confusion Matrix

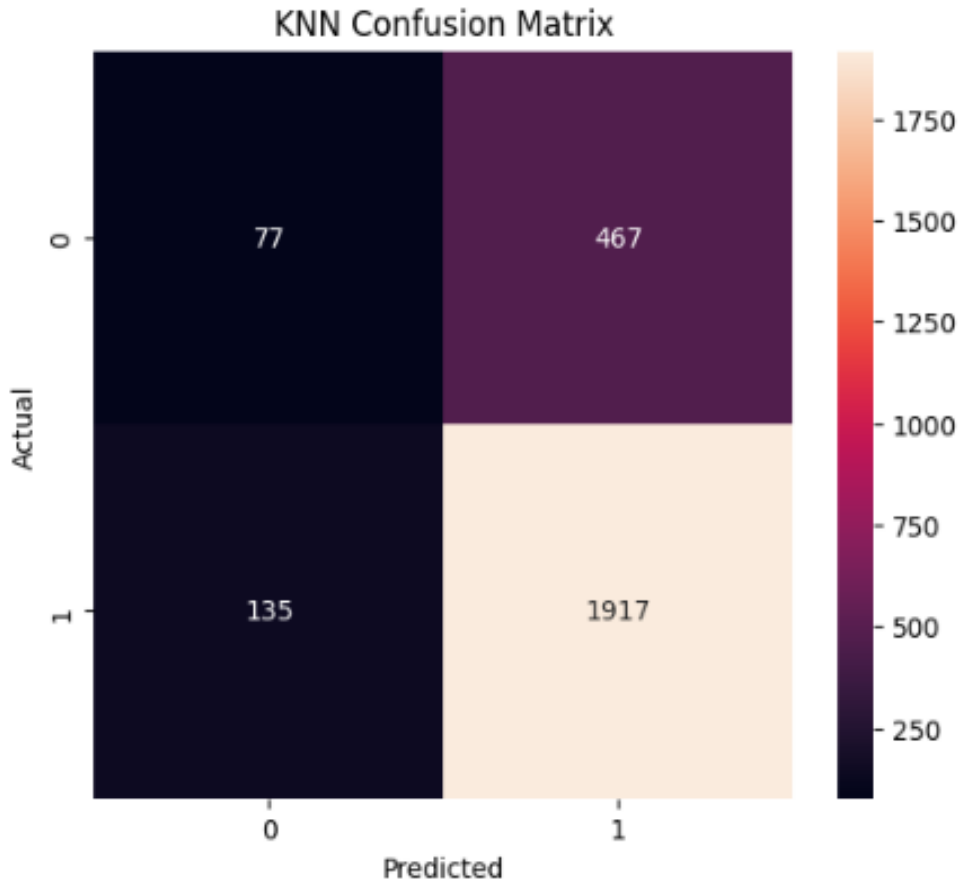
```
cm = confusion_matrix(y_test, y_pred_knn)

plt.figure(figsize=(6, 5))

sns.heatmap(
    cm,
    annot=True,
    fmt="d"
)

plt.title("KNN Confusion Matrix")
plt.xlabel("Predicted")
plt.ylabel("Actual")

plt.show()
```



- **SVM**

```
svm_model = SVC(
    kernel="linear"
)

svm_model.fit(X_train_scaled, y_train)

y_pred_svm = svm_model.predict(X_test_scaled)
```

Evaluation

```
print("Accuracy:",
      accuracy_score(y_test, y_pred_svm))

print("Precision:",
      precision_score(y_test, y_pred_svm))

print("Recall:",
      recall_score(y_test, y_pred_svm))

print("F1 Score:",
      f1_score(y_test, y_pred_svm))

print("\nClassification Report:\n")

print(classification_report(
    y_test,
    y_pred_svm
))
```

```
Accuracy: 0.7904468412942989
Precision: 0.7904468412942989
Recall: 1.0
F1 Score: 0.882960413080895

Classification Report:

              precision    recall  f1-score   support

           0       0.00      0.00      0.00       544
           1       0.79      1.00      0.88      2052

    accuracy                           0.79      2596
   macro avg       0.40      0.50      0.44      2596
weighted avg       0.62      0.79      0.70      2596
```

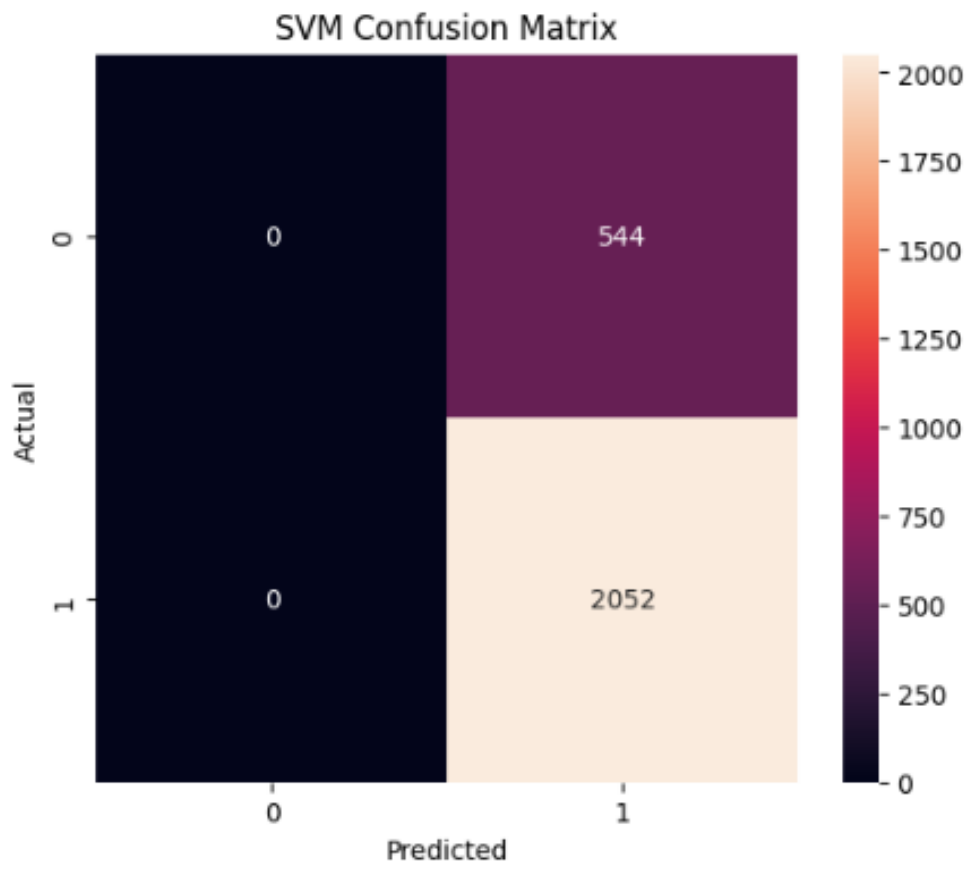
```
cm = confusion_matrix(y_test, y_pred_svm)

plt.figure(figsize=(6, 5))

sns.heatmap(
    cm,
    annot=True,
    fmt="d"
)

plt.title("SVM Confusion Matrix")
plt.xlabel("Predicted")
plt.ylabel("Actual")

plt.show()
```



- **Naive Byes**

```
nb_model = GaussianNB()

nb_model.fit(X_train_scaled, y_train)

y_pred_nb = nb_model.predict(X_test_scaled)
```

```
Evaluation
```

```
print("Accuracy:",
      accuracy_score(y_test, y_pred_nb))

print("Precision:",
      precision_score(y_test, y_pred_nb))

print("Recall:",
      recall_score(y_test, y_pred_nb))

print("F1 Score:",
      f1_score(y_test, y_pred_nb))

print("\nClassification Report:\n")

print(classification_report(
    y_test,
    y_pred_nb
))
```

```
Accuracy: 0.8012326656394453
Precision: 0.8111831442463533
Recall: 0.9756335282651072
F1 Score: 0.8858407079646018

Classification Report:

              precision    recall  f1-score   support

           0       0.61      0.14      0.23       544
           1       0.81      0.98      0.89      2052

    accuracy                           0.80      2596
   macro avg       0.71      0.56      0.56      2596
weighted avg       0.77      0.80      0.75      2596
```

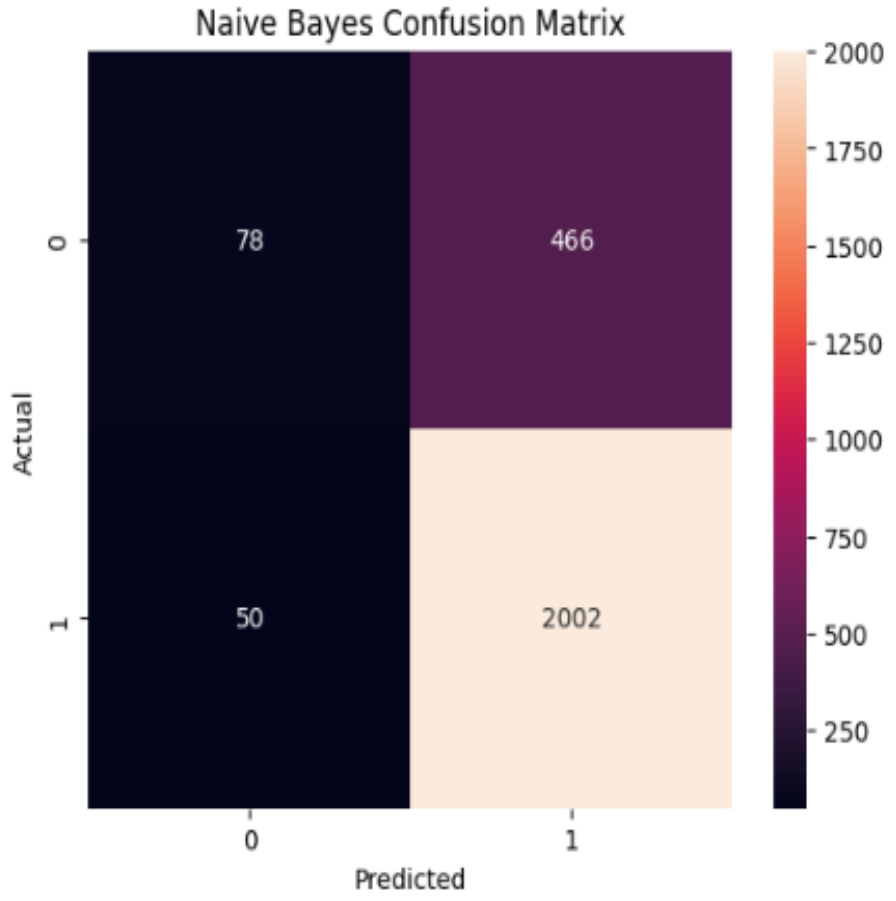
```
cm = confusion_matrix(y_test, y_pred_nb)

plt.figure(figsize=(6, 5))

sns.heatmap(
    cm,
    annot=True,
    fmt="d"
)

plt.title("Naive Bayes Confusion Matrix")
plt.xlabel("Predicted")
plt.ylabel("Actual")

plt.show()
```



- **Compare All Algorithms:**

```
results = pd.DataFrame({
    "Algorithm": [
        "Logistic Regression",
        "Decision Tree",
        "Random Forest",
        "KNN",
        "SVM",
        "Naive Bayes"
    ],
    "Accuracy": [
        accuracy_score(y_test, y_pred_logistic),
        accuracy_score(y_test, y_pred_dt),
        accuracy_score(y_test, y_pred_rf),
        accuracy_score(y_test, y_pred_knn),
        accuracy_score(y_test, y_pred_svm),
        accuracy_score(y_test, y_pred_nb)
    ],
    "Precision": [
        precision_score(y_test, y_pred_logistic),
        precision_score(y_test, y_pred_dt),
        precision_score(y_test, y_pred_rf),
        precision_score(y_test, y_pred_knn),
        precision_score(y_test, y_pred_svm),
        precision_score(y_test, y_pred_nb)
    ],
    "Recall": [
        recall_score(y_test, y_pred_logistic),
        recall_score(y_test, y_pred_dt),
```

```
        recall_score(y_test, y_pred_rf),
        recall_score(y_test, y_pred_knn),
        recall_score(y_test, y_pred_svm),
        recall_score(y_test, y_pred_nb)
    ],
    "F1 Score": [
        f1_score(y_test, y_pred_logistic),
        f1_score(y_test, y_pred_dt),
        f1_score(y_test, y_pred_rf),
        f1_score(y_test, y_pred_knn),
        f1_score(y_test, y_pred_svm),
        f1_score(y_test, y_pred_nb)
    ]
})
results = results.round(4)
print(results)
```

	Algorithm	Accuracy	Precision	Recall	F1 Score
0	Logistic Regression	0.7904	0.7904	1.0000	0.8830
1	Decision Tree	0.7018	0.8163	0.8036	0.8099
2	Random Forest	0.8220	0.8180	0.9966	0.8985
3	KNN	0.7681	0.8041	0.9342	0.8643
4	SVM	0.7904	0.7904	1.0000	0.8830
5	Naive Bayes	0.8012	0.8112	0.9756	0.8858

- **Accuracy Comparison Graph**

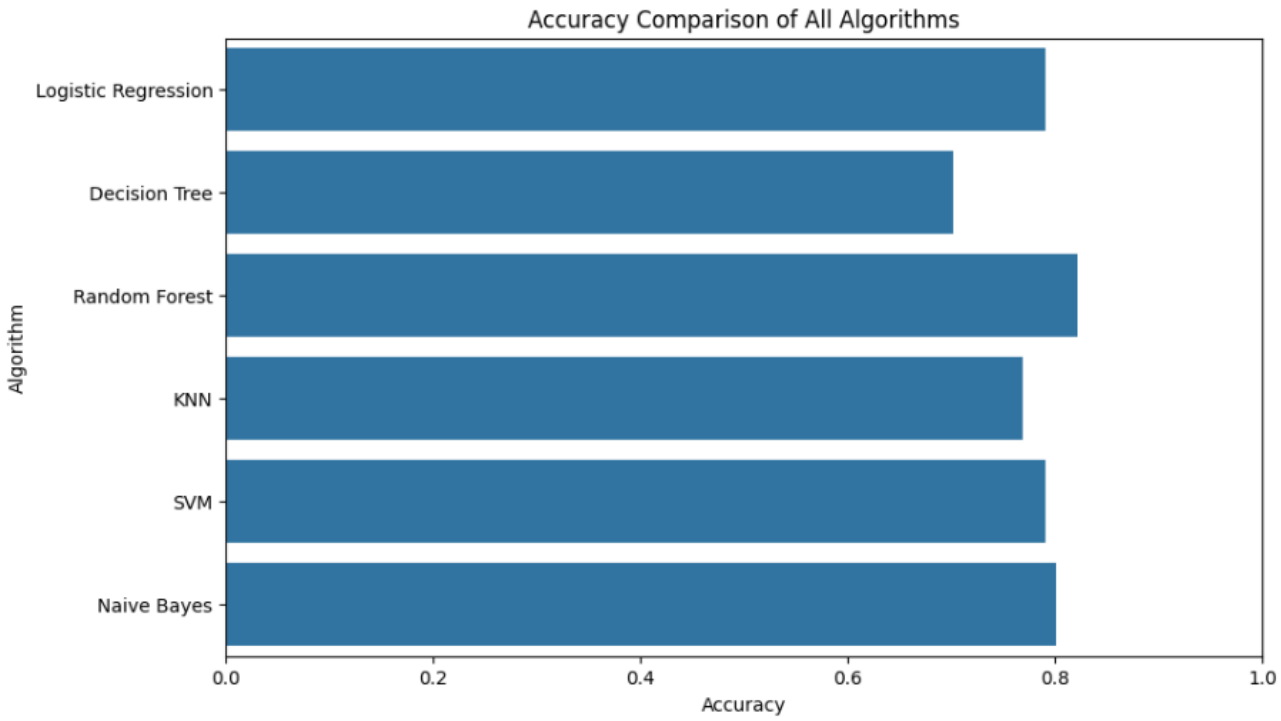
```
plt.figure(figsize=(10, 6))

sns.barplot(
    data=results,
    x="Accuracy",
    y="Algorithm"
)

plt.title("Accuracy Comparison of All Algorithms")

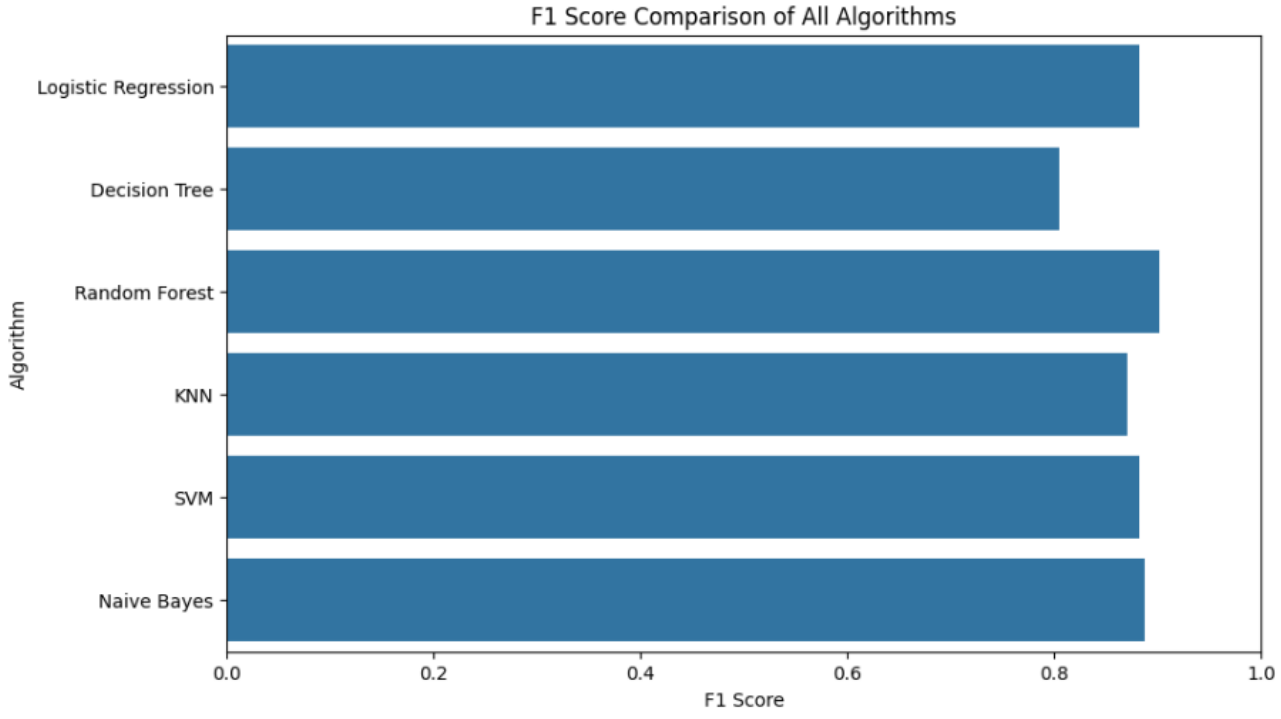
plt.xlim(0, 1)

plt.show()
```



- **F1-Score Comparison Graph**

```
plt.figure(figsize=(10, 6))
sns.barplot(
    data=results,
    x="F1 Score",
    y="Algorithm"
)
plt.title("F1 Score Comparison of All Algorithms")
plt.xlim(0, 1)
plt.show()
```



- **Find the Best Algorithm**

```
best_model = results.loc[
    results["F1 Score"].idxmax()
]

print("Best Model:")
print(best_model)
```

```
Best Model:
Algorithm    Random Forest
Accuracy            0.8278
Precision           0.8212
Recall              0.9994
F1 Score            0.9016
Name: 2, dtype: object
```

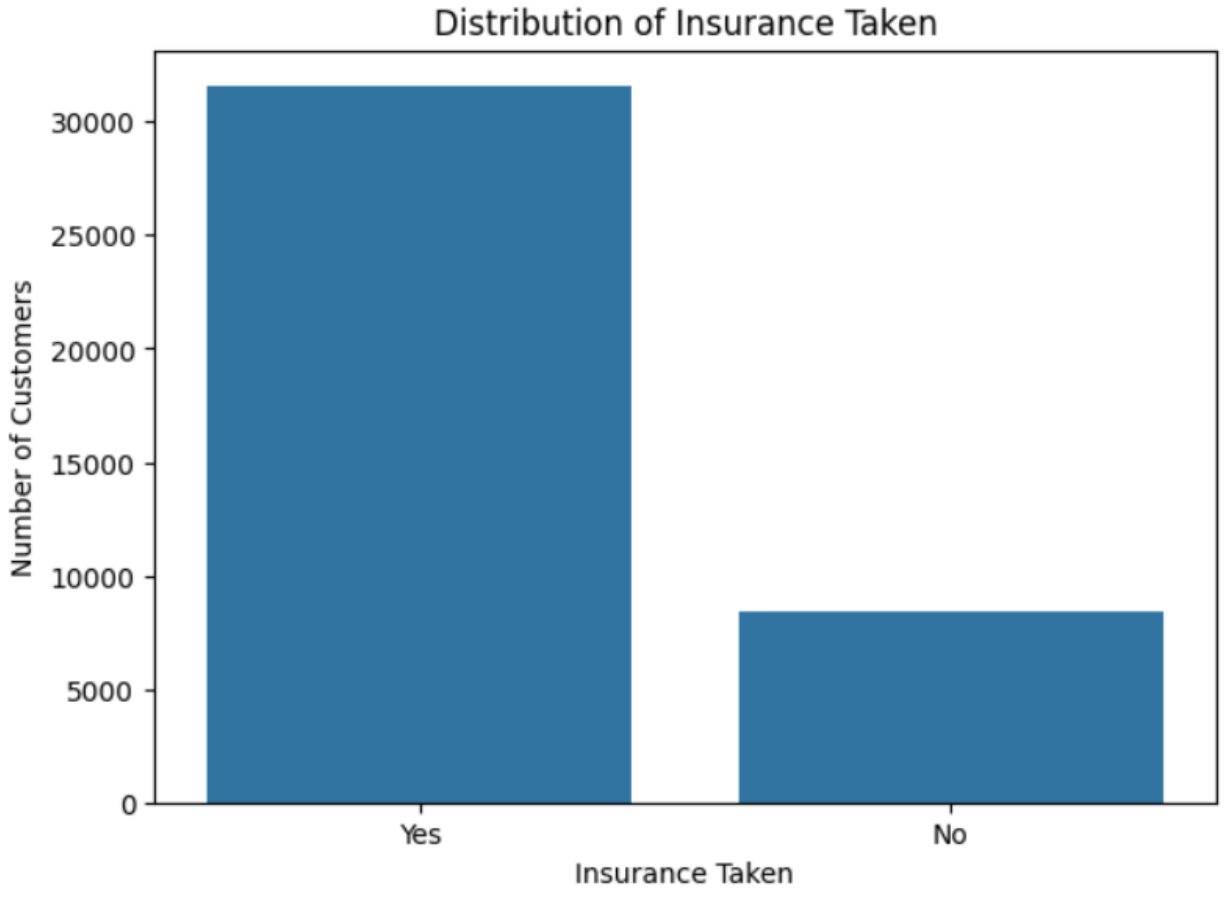
- **Problem Statement 1: Insurance Taken Distribution Graph:-Cont Plot**

```
plt.figure(figsize=(7, 5))

sns.countplot(
    data=df,
    x="Insurance_Taken"
)

plt.title("Distribution of Insurance Taken")
plt.xlabel("Insurance Taken")
plt.ylabel("Number of Customers")

plt.show()
```



- **Problem Statement 2: Analyze Occupation Distribution Graph :- Bar Graph**

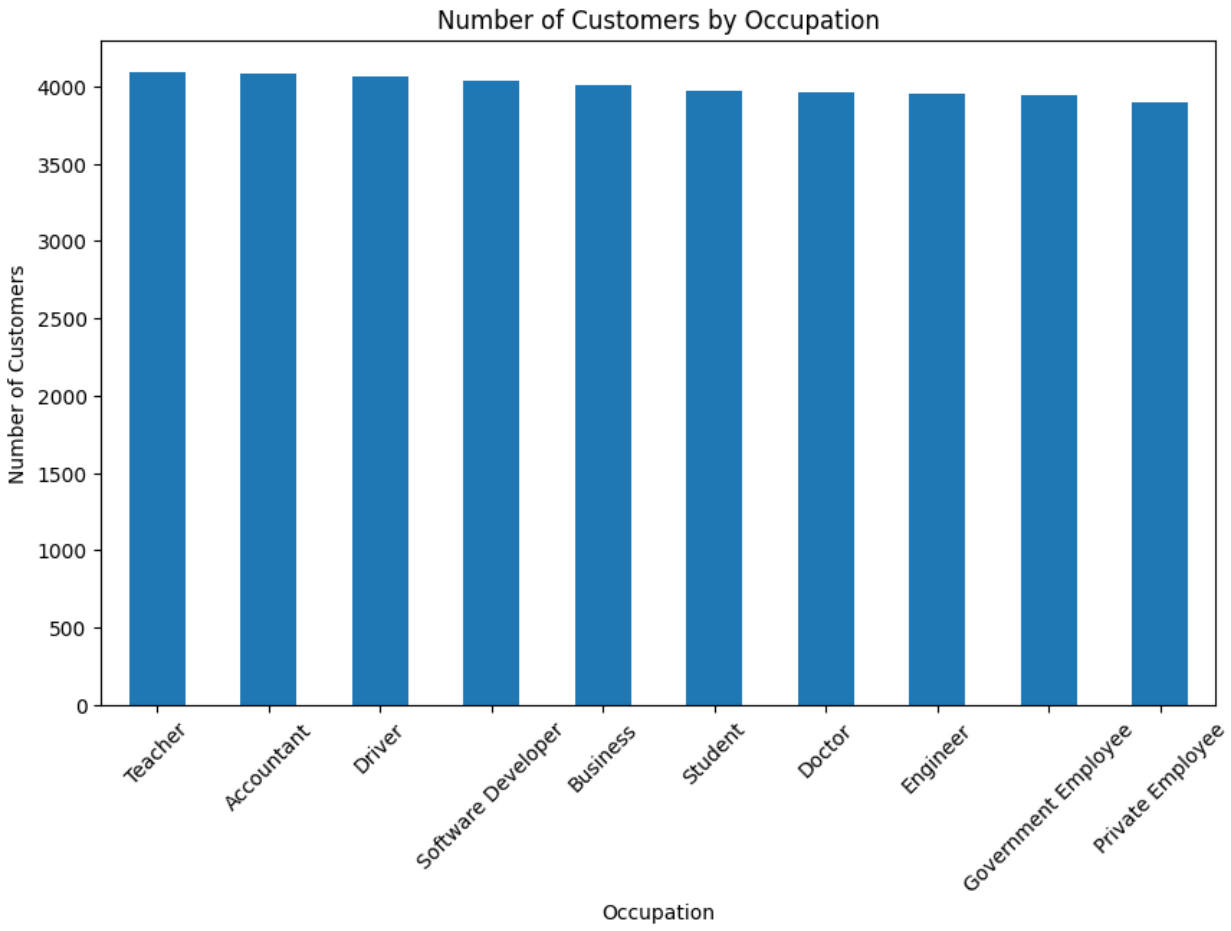
```
plt.figure(figsize=(10, 6))

df["Occupation"].value_counts().plot(
    kind="bar"
)

plt.title("Number of Customers by Occupation")
plt.xlabel("Occupation")
plt.ylabel("Number of Customers")

plt.xticks(rotation=45)

plt.show()
```



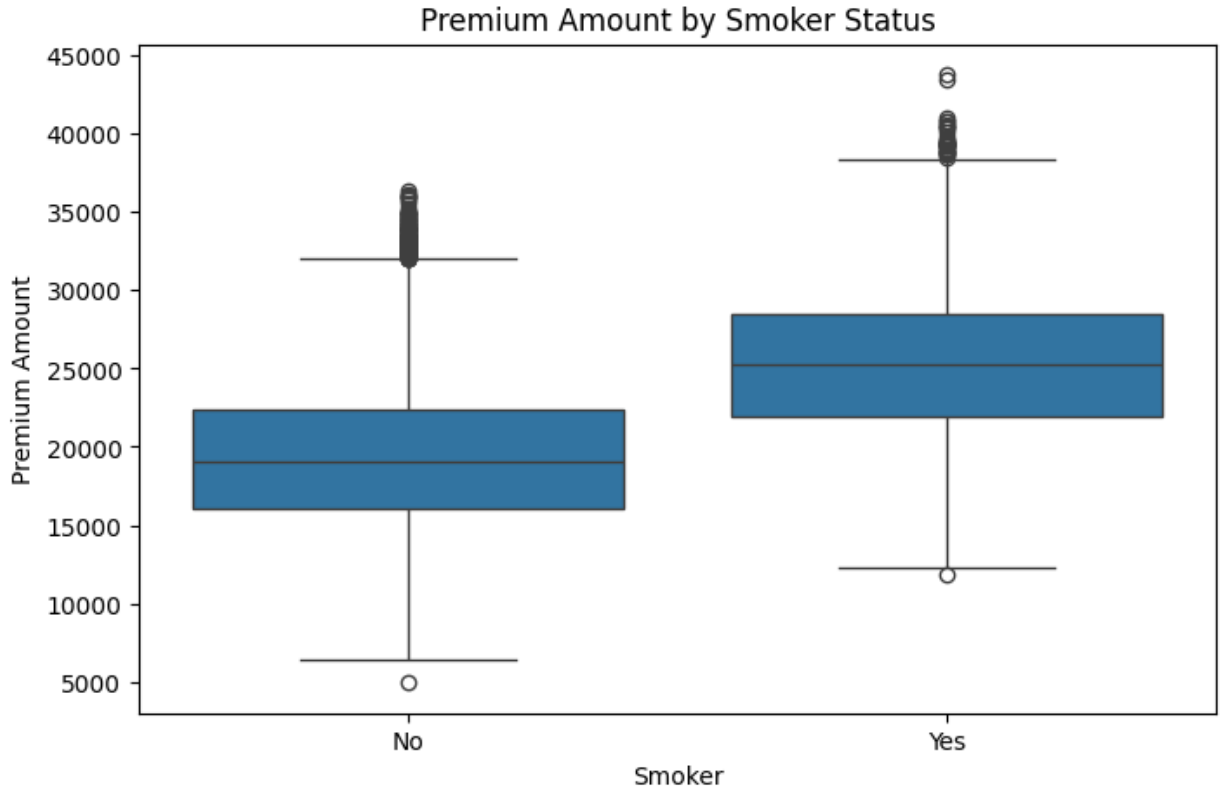
- **Problem Statement 3: Analyze Premium Amount by Smoker Status . Graph - BOX PLOT**

```
plt.figure(figsize=(8, 5))

sns.boxplot(
    data=df,
    x="Smoker",
    y="Premium_Amount"
)

plt.title("Premium Amount by Smoker Status")
plt.xlabel("Smoker")
plt.ylabel("Premium Amount")

plt.show()
```



- **Problem Statement :- 4 Analyze the percentage distribution of customers based on their marital status. Graph :- Pie Chart**

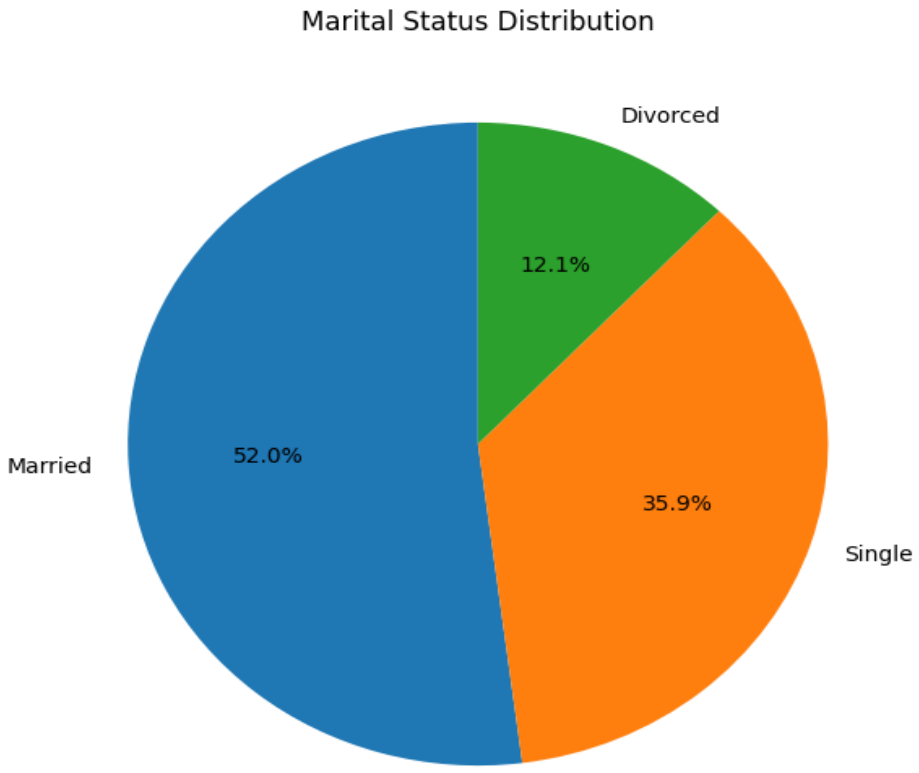
```
marital_count = df["Marital_Status"].value_counts()

plt.figure(figsize=(7, 7))

plt.pie(
    marital_count.values,
    labels=marital_count.index,
    autopct="%1.1f%%",
    startangle=90
)

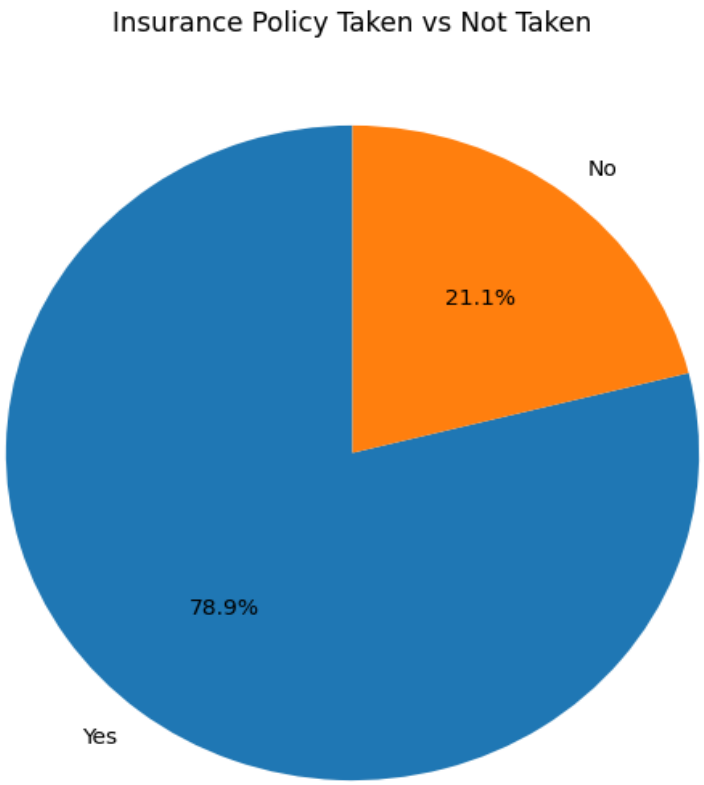
plt.title("Marital Status Distribution")

plt.show()
```



- **Problem Statement :-5 Analyze the percentage of customers who have taken an insurance policy and those who have not taken a policy.**

```
policy_count = df["Insurance_Taken"].value_counts()
plt.figure(figsize=(7, 7))
plt.pie(
    policy_count.values,
    labels=policy_count.index,
    autopct="%1.1f%%",
    startangle=90
)
plt.title("Insurance Policy Taken vs Not Taken")
plt.show()
```



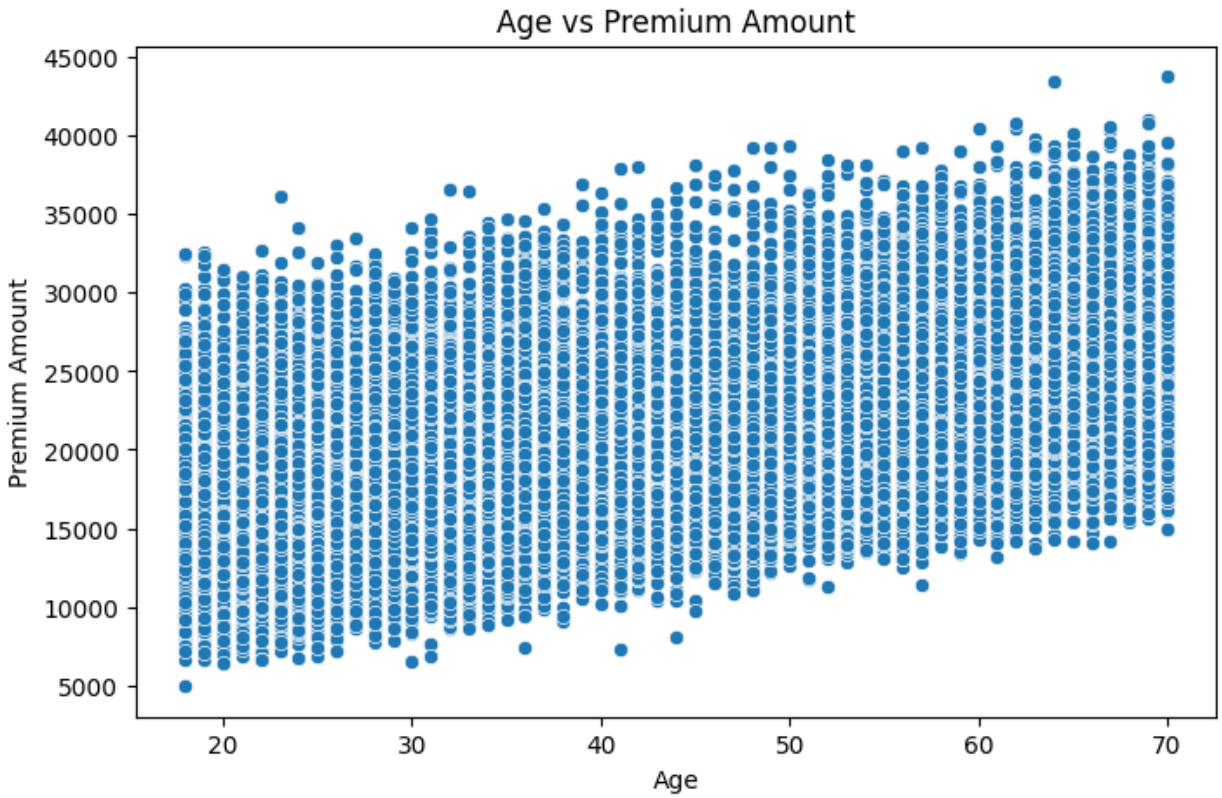
- **Problem Statement 6: Age vs Premium Amount**

```
plt.figure(figsize=(8, 5))

sns.scatterplot(
    data=df,
    x="Age",
    y="Premium_Amount"
)

plt.title("Age vs Premium Amount")
plt.xlabel("Age")
plt.ylabel("Premium Amount")

plt.show()
```



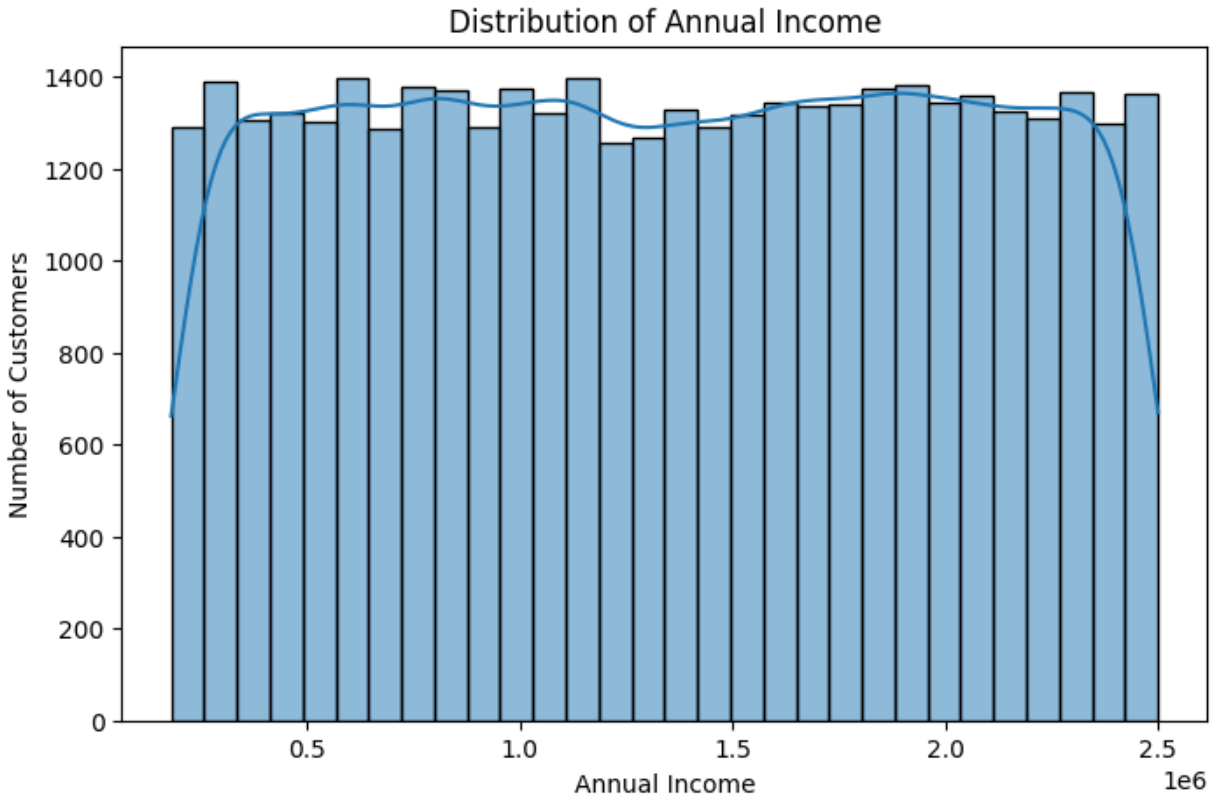
- **Problem Statement 7: Analyze Annual Income Distribution. Graph :- Histogram**

```
plt.figure(figsize=(8, 5))

sns.histplot(
    data=df,
    x="Annual_Income",
    bins=30,
    kde=True
)

plt.title("Distribution of Annual Income")
plt.xlabel("Annual Income")
plt.ylabel("Number of Customers")

plt.show()
```



4.3 Test Reports (Accuracy, Classification Reports):

Each model was evaluated using Confusion Matrix, Accuracy Score, and Classification Report.

Results include precision, recall, F1-score, and overall accuracy. Below are some sample findings:

Sr. No.	Algorithm	Accuracy	Precision	Recall	F1 Score
0	Logistic Regression	0.7892	0.7892	1.0000	0.8822
1	Decision Tree	0.6991	0.8236	0.7875	0.8051
2	Random Forest	0.8278	0.8212	0.9994	0.9016
3	KNN	0.7785	0.8081	0.9435	0.8705
4	SVM	0.7892	0.7892	1.0000	0.8822
5	Naive Bayes	0.8029	0.8049	0.9902	0.8880

5. Proposed Enhancements:

The following enhancements can be proposed for the Insurance Policy Prediction Using Machine Learning project:

Use More Customer Features:
Additional features such as customer lifestyle, previous insurance history, family medical history, and financial information can be included to improve prediction performance.

Hyperparameter Tuning:
The performance of machine learning algorithms can be improved by optimizing their parameters using techniques such as Grid Search or Random Search.

Advanced Machine Learning Algorithms:
Additional classification algorithms and advanced ensemble models can be implemented and compared with the existing models.

Interactive Dashboard:
An interactive dashboard can be developed using Power BI or Tableau to display customer insights, insurance trends, and model performance.

Real-Time Prediction System:
The trained model can be integrated into a web application where users can enter customer information and receive an insurance policy prediction.

Automated Model Selection:
The system can be enhanced to automatically compare multiple algorithms and select the best-performing model based on evaluation metrics.

Regular Model Updates:
The model can be retrained periodically with new customer data to maintain and improve its prediction performance.

These enhancements can make the proposed system more accurate, practical, user-friendly, and suitable for real-world insurance applications.

6. Conclusion:

The Insurance Policy Prediction Using Machine Learning project was developed to predict whether a customer is likely to take an insurance policy based on customer-related information. The dataset was analyzed and preprocessed by checking missing values and duplicate records, removing the data leakage column, encoding categorical data, splitting the dataset into training and testing data, and applying feature scaling where required.

Multiple classification algorithms, including Logistic Regression, Decision Tree, Random Forest, KNN, SVM, and Naive Bayes**,** were trained and evaluated using Accuracy, Precision, Recall, F1-Score, Classification Report, and Confusion Matrix. The project demonstrates how machine learning can analyze insurance customer data, identify patterns, compare different algorithms, and select the best-performing model for insurance policy prediction.

7. Bibliography:

1. scikit-learn: Machine Learning in Python – https://scikit-learn.org/

2. pandas Documentation – https://pandas.pydata.org/

3. seaborn Documentation – https://seaborn.pydata.org/

4. matplotlib Documentation – https://matplotlib.org/

5. Heart Disease Dataset (original inspiration) – https://www.kaggle.com/